1) 차원 축소란**?**

- 많은 특성(Feature)으로 구성된 고차원 데이터의 차원을 줄여 새로운 저차원 데이터로 변환하는 방법
- 데이터가 가진 중요한 정보와 특성은 최대한 유지하면서 데이터의 복잡성을 줄이는 것이 목적
- 단순히 특성의 개수를 줄이는 것뿐 아니라, 데이터를 설명할 수 있는 잠재적인 요소(Latent Factor)를 추출하는 의미도 있음

2) 차원 축소가 필요한 이유

- 고차원 데이터의 희소성: 차원이 증가할수록 데이터 포인트 사이의 거리가 기하급수적으로 멀어지고 데이터가 희소한 구조를 가질 수 있음
- 예측 신뢰도 저하: 수백 개 이상의 특성으로 구성된 데이터는 상대적으로 적은 차원에서 학습된 모델보다 예측 신뢰도가 떨어질 수 있음
- 다중 공선성(Multicollinearity): 특성의 개수가 많아지면 특성 간 상관관계가 높아질 가능성이 커짐. 선형 회귀와 같은 선형 모델에서는 입력 변수 간 높은 상관관계가 예측 성능을 저하시킬 수 있음
- 데이터 해석의 어려움: 특성이 많으면 데이터를 시각적으로 표현하거나 전체적인 특징을 파악하기 어려움
- 학습 비용 증가: 데이터의 차원을 줄이면 학습 데이터의 크기가 감소하여 학습에 필요한 처리 능력과 계산 비용을 줄일 수 있음

3) 차원 축소의 두 가지 방법

① 특성 선택(Feature Selection)

- 기존 특성 중 데이터의 특징을 잘 나타내는 주요 특성만 선택하는 방법
- 불필요하거나 특정 특성에 종속성이 강한 특성을 제거함
- 기존 특성을 그대로 사용하기 때문에 결과를 해석하기 쉬움

② 특성 추출(Feature Extraction)

- 기존 특성들을 조합하거나 변환하여 새로운 특성을 추출하는 방법
- 기존 특성을 압축하고 새로운 공간에 매핑하여 함축적인 특성을 생성함
- 추출된 특성은 기존 특성과 다른 새로운 값으로 구성됨
- 기존 데이터만으로 직접 파악하기 어려운 잠재적인 요소(Latent Factor)를 찾아낼 수 있음

4) 잠재적 요소**(Latent Factor)**

- 데이터에 직접적으로 드러나지는 않지만 여러 특성에 내재되어 있는 숨겨진 요소
- 여러 개의 기존 특성을 분석하여 데이터의 핵심적인 의미를 함축적으로 표현한 것
- 예를 들어 학생의 모의고사 성적, 내신 성적, 봉사활동, 수상 경력 등의 정보를 학업 성취도나 의사소통 능력과 같은 요소로 요약할 수 있음
- PCA, SVD, NMF 등의 차원 축소 알고리즘을 통해 추출할 수 있음

5) 차원 축소의 활용 분야

① 이미지 데이터

- 이미지 데이터는 매우 많은 픽셀로 구성되어 있어 차원이 높음
- 차원 축소를 통해 이미지의 핵심적인 특징을 추출하고 압축할 수 있음
- 이미지 분류 과정에서 불필요한 정보의 영향을 줄이고 과적합(Overfitting)을 완화하는 데 도움을 줌
- 이미지 인식, 패턴 인식, 이미지 압축 등에 활용됨

② 텍스트 데이터

- 텍스트 문서는 많은 단어로 구성되어 있음
- 단어의 구성과 사용 패턴을 분석하여 문서에 숨겨진 의미(Semantic)와 주제(Topic)를 추출할 수 있음
- SVD와 NMF가 텍스트의 잠재적인 의미와 토픽을 분석하는 데 활용됨

2. PCA(Principal Component Analysis, 주성분 분석)

1) PCA란?

- 대표적인 차원 축소 기법
- 여러 변수 사이에 존재하는 상관관계를 이용하여 데이터를 대표하는 주성분(Principal Component)을 추출하는 방법
- 기존 데이터의 정보 손실을 최소화하면서 차원을 축소하는 것이 목적
- 데이터의 분산이 가장 큰 방향을 찾아 새로운 축으로 설정하고, 해당 축에 원본 데이터를 투영함

2) 주성분(Principal Component)

- 기존 특성들의 정보를 조합하여 새롭게 생성한 변수
- 원본 데이터의 분산을 최대한 설명할 수 있는 방향으로 구성됨
- 첫 번째 주성분은 데이터의 변동성을 가장 많이 설명하는 방향임
- 두 번째 주성분은 첫 번째 주성분과 직각을 이루면서 남은 변동성을 가장 많이 설명하는 방향임
- 이후 주성분도 앞선 주성분과 직교하는 방향으로 생성됨
- 주성분을 적절하게 선택하면 적은 수의 특성으로 원본 데이터의 변동성을 상당 부분 설명할 수 있음

3) PCA의 주요 개념

① 분산(Variance)

- 하나의 변수가 평균을 중심으로 얼마나 퍼져 있는지를 나타내는 값
- 분산이 클수록 데이터가 넓게 흩어져 있다는 의미
- PCA는 데이터의 분산이 큰 방향을 중요한 정보를 많이 포함한 방향으로 판단함

② 공분산(Covariance)

- 두 변수가 함께 변화하는 정도를 나타내는 값
- 양의 공분산: 한 변수가 증가할 때 다른 변수도 증가하는 경향
- 음의 공분산: 한 변수가 증가할 때 다른 변수는 감소하는 경향
- 공분산이 0에 가까우면 두 변수 사이의 선형적인 관계가 약하다는 의미

③ 공분산 행렬(Covariance Matrix)

- 여러 변수 사이의 공분산을 행렬 형태로 표현한 것
- 변수 간의 관계와 데이터의 분산 구조를 파악할 수 있음
- 대각선 원소는 각 변수의 분산을 나타냄
- 대각선 이외의 원소는 변수 사이의 공분산을 나타냄
- PCA에서 주성분을 계산하는 기초 자료로 활용됨

④ 고유값(Eigenvalue)

- 특정 방향에서 데이터가 가지는 분산의 크기를 나타내는 값
- 고유값이 클수록 해당 방향이 데이터의 분산을 많이 설명함
- 고유값이 큰 순서대로 주성분을 선택함

⑤ 고유벡터(Eigenvector)

- 행렬을 곱해도 방향은 변하지 않고 크기만 변하는 벡터
- 수식으로는 $Ax = \lambda x$로 표현할 수 있음
- A는 행렬, x는 고유벡터, λ는 고유값을 의미함
- PCA에서는 고유벡터가 주성분의 방향을 나타냄

⑥ 선형 변환(Linear Transformation)

- 특정 벡터에 행렬을 곱하여 새로운 벡터로 변환하는 과정
- 벡터를 기존 공간에서 새로운 공간으로 투영하는 개념으로 이해할 수 있음
- PCA는 고유벡터를 이용해 입력 데이터를 새로운 공간으로 선형 변환함

⑦ 정방행렬(Square Matrix)

- 행의 개수와 열의 개수가 같은 행렬
- 예를 들어 2×2, 3×3 행렬이 이에 해당함
- PCA에서 사용하는 공분산 행렬은 정방행렬임

⑧ 대칭행렬(Symmetric Matrix)

- 주대각선을 기준으로 서로 대칭되는 위치의 원소 값이 같은 행렬
- 공분산 행렬은 대칭행렬의 특징을 가짐
- 대칭행렬은 고유값과 고유벡터를 이용해 분해할 수 있으며, 고유벡터는 서로 직교하는 특성을 가짐

⑨ 직교행렬(Orthogonal Matrix)

- 행렬을 구성하는 벡터들이 서로 직교하는 행렬
- PCA에서 공분산 행렬을 고유값과 고유벡터를 이용해 분해할 때 활용되는 개념
- 주성분 사이의 직교 관계와 연결됨

4) PCA의 수행 과정

1. 입력 데이터 세트의 공분산 행렬을 생성함.
2. 공분산 행렬의 고유벡터와 고유값을 계산함.
3. 고유값이 큰 순서대로 PCA 변환 차원 수(K)에 해당하는 고유벡터를 선택함.
4. 선택한 고유벡터를 이용해 원본 데이터를 새로운 차원으로 변환함.

5) 설명된 분산 비율(Explained Variance Ratio)

- 전체 데이터의 분산 중 각 주성분이 차지하는 비율
- 각 주성분이 원본 데이터의 정보를 얼마나 설명하는지 확인할 수 있음
- 비율이 높은 주성분을 선택하면 원본 데이터의 정보를 더 많이 유지할 수 있음
- PCA에서 주성분의 개수를 결정하는 기준으로 활용됨

6) 데이터 스케일링(Scaling)

- 특성마다 값의 범위와 단위가 다르면 특정 특성이 PCA 결과에 더 큰 영향을 미칠 수 있음
- PCA는 여러 특성의 값을 함께 계산하므로 분석 전에 특성의 스케일을 맞추는 것이 중요함

StandardScaler

- 데이터의 평균을 0, 분산을 1로 변환하는 표준화 방법
- 서로 다른 범위와 단위를 가진 특성을 동일한 기준에서 분석할 수 있도록 함
- PCA를 적용하기 전에 자주 사용됨

7) PCA의 특징

- 비지도학습(Unsupervised Learning) 방식임
- 정답 레이블 없이 데이터 자체의 분산 구조를 이용함
- 데이터의 분산이 큰 방향을 중심으로 주성분을 추출함
- 주성분들은 서로 직교함
- 이미지 압축과 얼굴 인식 등의 컴퓨터 비전 분야에 활용됨
- PCA를 이용한 얼굴 인식 방식 중 하나로 Eigen-face가 있음

8) PCA 실습에서 알아둘 내용

- 사이킷런에서는 `PCA` 클래스를 이용함
- `n_components`: 변환할 주성분의 개수를 지정하는 파라미터
- `fit()`: 입력 데이터의 주성분을 학습하는 메서드
- `transform()`: 학습한 주성분을 이용해 데이터를 변환하는 메서드
- `fit_transform()`: 학습과 데이터 변환을 한 번에 수행하는 메서드
- `explained_variance_ratio_`: 각 주성분의 설명된 분산 비율을 확인하는 속성

붓꽃(Iris) 데이터 실습

- 원본 데이터는 꽃받침 길이와 너비, 꽃잎 길이와 너비 등 4개의 특성으로 구성됨

- StandardScaler로 데이터를 표준화한 뒤 PCA를 적용함
- 4개의 특성을 2개의 주성분으로 축소함
- 두 주성분의 설명된 분산 비율은 약 72.9%, 22.8%로 나타남
- 2개의 주성분만으로 원본 데이터의 변동성을 약 95.7% 설명할 수 있음

신용카드 고객 데이터 실습

- 30,000개의 데이터와 24개의 속성으로 구성된 데이터 사용
- 타깃을 제외한 23개 특성으로 연체 여부를 예측함
- 청구 금액 관련 특성(BILL_AMT1~BILL_AMT6)은 서로 높은 상관관계를 가짐
- 해당 6개 특성을 2개의 PCA 컴포넌트로 축소했을 때 약 95% 이상의 변동성을 설명함
- 전체 23개 특성을 6개의 주성분으로 축소했을 때 원본 데이터보다 예측 정확도가 약간 낮아졌지만, 특성 개수를 크게 줄일 수 있었음

3. LDA(Linear Discriminant Analysis, 선형 판별 분석)

1) LDA란?

- 선형 판별 분석이라고 불리는 차원 축소 기법
- 입력 데이터를 저차원 공간으로 투영하여 차원을 축소함
- PCA와 달리 데이터의 결정 클래스(Class)를 최대한 분리할 수 있는 축을 찾는 것이 목적
- 분류(Classification)에 활용하기 적합하도록 클래스 간의 구분을 최대화함
- 지도학습(Supervised Learning) 방식임

2) PCA와 LDA의 차이

- PCA: 입력 데이터의 변동성이 가장 큰 축을 찾음
- LDA: 입력 데이터의 결정 클래스를 최대한 분리할 수 있는 축을 찾음
- PCA는 정답 레이블을 사용하지 않지만, LDA는 클래스 정보를 사용함
- PCA는 데이터의 전체적인 분산을 중요하게 생각하고, LDA는 클래스 간 구분을 중요하게 생각함

3) LDA의 주요 개념

① 클래스(Class)

- 데이터가 속하는 집단 또는 범주
- 예를 들어 붓꽃 데이터에서 Setosa, Versicolor, Virginica가 각각 하나의 클래스임

② 클래스 간 분산(Between-Class Scatter)

- 서로 다른 클래스의 평균이 얼마나 떨어져 있는지를 나타냄
- 클래스 간 분산이 클수록 서로 다른 클래스가 멀리 분리됨
- LDA에서는 클래스 간 분산을 최대화하는 것이 중요함

③ 클래스 내부 분산(Within-Class Scatter)

- 같은 클래스에 속한 데이터들이 얼마나 흩어져 있는지를 나타냄
- 클래스 내부 분산이 작을수록 같은 클래스의 데이터들이 서로 가깝게 모여 있음
- LDA에서는 클래스 내부 분산을 최소화하는 것이 중요함

④ 평균 벡터(Mean Vector)

- 특정 클래스에 속한 데이터들의 특성별 평균값을 나타내는 벡터
- 클래스 간 분산과 클래스 내부 분산을 계산할 때 사용됨

4) LDA의 핵심 원리

- 클래스 간 분산을 최대화함
- 클래스 내부 분산을 최소화함
- 서로 다른 클래스는 최대한 멀리 떨어지도록 함
- 같은 클래스에 속하는 데이터는 최대한 가깝게 모이도록 함
- 결과적으로 새로운 공간에서 각 클래스가 잘 구분되도록 차원을 축소함

5) LDA의 수행 과정

1. 각 클래스의 평균 벡터를 계산하고 클래스 내부 분산 행렬과 클래스 간 분산 행렬을 구함.
2. 두 분산 행렬을 바탕으로 고유값과 고유벡터를 계산함.
3. 고유값이 큰 순서대로 변환 차원 수(K)에 해당하는 고유벡터를 선택함.
4. 선택한 고유벡터를 이용해 입력 데이터를 새로운 차원으로 변환함.

6) LDA의 특징

- 지도학습 방식으로 정답 레이블이 필요함
- 클래스 정보를 활용하여 차원을 축소함
- 클래스 간 분산을 크게 하고 클래스 내부 분산을 작게 하는 방향을 찾음
- 분류 문제에서 데이터의 구분을 명확하게 하는 데 유용함
- 사이킷런에서는 `LinearDiscriminantAnalysis` 클래스를 사용함

7) LDA 실습에서 알아둘 내용

- `n_components`: 변환할 차원의 개수를 지정함
- `fit()`: 입력 데이터와 정답 레이블을 이용해 학습함
- `transform()`: 학습된 판별 방향을 이용해 데이터를 변환함
- PCA와 달리 `fit()` 과정에서 정답 레이블을 함께 입력해야 함

4. SVD(Singular Value Decomposition, 특이값 분해)

1) SVD란?

- 행렬을 여러 개의 행렬로 분해하는 기법
- 특이값 분해라고 불림

- PCA와 유사하게 행렬 분해를 이용하지만, 정방행렬뿐 아니라 행과 열의 크기가 다른 행렬에도 적용할 수 있음
- 고차원 행렬을 저차원 행렬로 분해하여 데이터의 핵심적인 정보를 추출할 수 있음
- 차원 축소, 데이터 압축, 잠재적 요소 추출 등에 활용됨

2) SVD의 기본 수식

$$A = U\Sigma V^T$$

- A: 원본 행렬
- U: 왼쪽 특이벡터로 구성된 행렬
- Σ: 특이값을 포함하는 대각행렬
- V^T: 오른쪽 특이벡터로 구성된 행렬의 전치행렬

3) SVD의 주요 개념

① 특이벡터(Singular Vector)

- SVD를 통해 분해된 행렬 U와 V에 포함된 벡터
- 특이벡터들은 서로 직교하는 특성을 가짐
- 원본 행렬을 구성하는 방향과 관련된 정보를 나타냄

② 특이값(Singular Value)

- Σ 행렬의 대각선에 위치하는 값
- 각 특이벡터가 원본 데이터의 정보를 얼마나 설명하는지 나타냄
- 특이값이 클수록 해당 요소가 원본 행렬을 설명하는 데 중요한 역할을 함
- 특이값이 0인 경우 해당 방향에 대응하는 정보가 없음을 의미함

③ 대각행렬(Diagonal Matrix)

- 주대각선에 있는 원소를 제외한 나머지 원소가 모두 0인 행렬
- SVD에서 Σ 행렬이 이에 해당함
- 특이값은 대각선에 배치됨

④ 전치행렬(Transpose)

- 기존 행렬의 행과 열을 서로 바꾼 행렬
- V의 전치행렬을 V^T로 표현함
- SVD의 기본 분해식에 사용됨

⑤ 행렬 분해(Matrix Factorization)

- 하나의 행렬을 여러 개의 행렬로 나누어 표현하는 방법
- 원본 데이터의 구조를 더 작은 행렬들로 표현할 수 있음
- SVD와 NMF가 대표적인 행렬 분해 기법임

⑥ 선형 독립(Linear Independence)

- 여러 벡터 중 어느 하나도 다른 벡터들의 선형 결합으로 표현할 수 없는 관계
- 각 벡터가 서로 독립적인 정보를 가진다는 의미로 이해할 수 있음

⑦ 선형 종속(Linear Dependence)

- 하나의 벡터가 다른 벡터들의 선형 결합으로 표현될 수 있는 관계
- 데이터에 중복되거나 의존적인 정보가 존재하는 경우와 관련됨

⑧ 랭크(Rank)

- 행렬에서 선형 독립인 행 또는 열 벡터의 최대 개수
- 행렬이 가지는 독립적인 정보의 차원을 나타냄
- 랭크가 작을수록 행렬을 더 낮은 차원의 구조로 표현할 수 있음
- SVD에서 0이 아닌 특이값의 개수는 행렬의 랭크와 같음

4) SVD의 수행과 행렬 복원

- 원본 행렬을 U, Σ, V^T로 분해함
- 분해된 세 행렬을 다시 곱하면 원본 행렬을 복원할 수 있음
- 특이값이 0인 부분에 대응하는 행렬 요소는 제거해도 원본 행렬을 복원하는 데 영향을 주지 않음
- 선형 종속성이 높은 데이터에서는 0인 특이값이 나타날 수 있음
- 0이 아닌 특이값과 이에 대응하는 특이벡터를 이용해 행렬을 더 작은 크기로 표현할 수 있음

5) Truncated SVD(절단된 특이값 분해)

① Truncated SVD란?

- SVD에서 특이값이 큰 상위 일부만 선택하여 행렬을 분해하는 방법
- 작은 특이값에 대응하는 요소들을 제거하여 행렬의 차원을 더 줄임
- 원본 데이터의 중요한 정보는 유지하면서 데이터 크기를 압축할 수 있음

② 일반 SVD와의 차이

- 일반 SVD: 전체 특이값과 특이벡터를 이용해 원본 행렬을 분해함
- Truncated SVD: 상위 일부 특이값과 특이벡터만 선택하여 분해함
- Truncated SVD는 원본 행렬을 완벽하게 복원하지 못할 수 있음
- 대신 원본 데이터와 상당히 유사한 형태로 근사 복원할 수 있음
- 선택하는 특이값의 개수가 많을수록 일반적으로 원본 행렬에 가까워짐

③ 저랭크 근사(Low-Rank Approximation)

- 원본 행렬보다 낮은 랭크를 가진 행렬로 원본 데이터를 근사하는 방법
- 중요한 정보만 남기고 상대적으로 중요도가 낮은 정보를 줄임
- Truncated SVD와 NMF 등에서 활용되는 핵심 개념임

6) 희소 행렬과 밀집 행렬

① 밀집 행렬(Dense Matrix)

- 행렬의 대부분 원소가 0이 아닌 값을 가지는 행렬

② 희소 행렬(Sparse Matrix)

- 행렬의 원소 중 대부분이 0인 행렬
- 텍스트 데이터처럼 특성의 개수가 많지만 실제 값이 존재하는 부분은 적은 데이터에서 나타날 수 있음

③ PCA와 SVD의 차이

- 책에서는 PCA가 밀집 행렬에 대한 변환만 가능한 반면, SVD는 희소 행렬에 대한 변환도 가능하다고 설명함
- 따라서 희소 행렬로 표현되는 텍스트 데이터 등의 처리에서 Truncated SVD가 유용하게 사용됨

7) 사이킷런 **TruncatedSVD** 클래스

- 사이킷런에서는 `TruncatedSVD` 클래스를 이용함
- `n_components`: 추출할 주요 컴포넌트의 개수를 지정함
- `fit()`: 입력 데이터에 대해 학습함
- `transform()`: 학습된 컴포넌트를 이용해 데이터를 변환함
- PCA와 유사한 방식으로 원본 데이터를 저차원으로 변환할 수 있음

8) **SVD**의 활용 분야

① 이미지 압축 및 패턴 인식

- 이미지의 핵심적인 정보를 추출하여 압축하는 데 활용됨
- 이미지의 패턴을 인식하는 컴퓨터 비전 분야에서 사용됨

② 신호 처리

- 데이터의 주요한 구조를 추출하고 압축하는 데 활용됨

③ 텍스트 분석

- 문서에 포함된 단어의 관계를 분석하여 숨겨진 의미와 주제를 추출함

④ LSA(Latent Semantic Analysis, 잠재 의미 분석)

- 문서와 단어 사이의 관계를 분석하여 문서에 숨겨진 의미를 추출하는 방법
- SVD를 기반으로 텍스트의 잠재적인 의미와 주제를 분석함
- 문서의 단어 구성이 비슷한 경우 문서 사이의 의미적 유사성을 파악하는 데 활용됨

5. NMF(Non-Negative Matrix Factorization, 비음수 행렬 분해)

1) NMF란?

- 비음수 행렬 분해라고 불리는 행렬 분해 기법
- 원본 행렬의 모든 원소가 0 이상이라는 조건에서 원본 행렬을 두 개의 양수 행렬로 분해하는 방법
- Truncated SVD와 마찬가지로 저랭크 근사(Low-Rank Approximation)를 이용하는 행렬 분해 방식
- 행렬 분해를 통해 데이터에 숨겨진 잠재적인 요소를 추출할 수 있음

2) NMF의 기본 원리

- 원본 행렬을 두 개의 작은 행렬로 분해함
- 두 행렬을 곱하여 원본 행렬을 근사적으로 표현함
- 원본 행렬의 모든 원소가 0 이상이어야 한다는 조건이 있음
- 분해된 행렬은 원본 데이터에 내재된 잠재적인 요소를 나타냄

3) NMF의 기본 수식

$$V \approx W \times H$$

- V: 원본 행렬
- W: 원본 데이터의 행과 잠재 요소 사이의 관계를 나타내는 행렬
- H: 잠재 요소가 원본 데이터의 특성을 어떻게 구성하는지 나타내는 행렬
- 원본 행렬보다 작은 크기의 두 행렬을 곱하여 원본 행렬을 근사함

4) NMF의 주요 개념

① 비음수(Non-Negative)

- 0보다 작은 값이 없는 상태
- NMF는 원본 행렬의 모든 원소가 0 이상이어야 한다는 조건을 가짐
- 분해된 행렬 역시 양수 값을 기반으로 구성됨

② 행렬 분해(Matrix Factorization)

- 하나의 큰 행렬을 여러 개의 작은 행렬로 나누는 방법
- 복잡한 데이터를 더 간결한 구조로 표현할 수 있음
- NMF에서는 원본 행렬을 두 개의 행렬로 분해함

③ 저랭크 근사(Low-Rank Approximation)

- 원본 행렬보다 낮은 랭크를 가진 행렬을 이용해 원본 데이터를 근사하는 방법
- 데이터의 중요한 정보와 구조를 유지하면서 차원을 축소함

④ 잠재 요소(Latent Factor)

- 데이터에 직접 드러나지 않지만 여러 특성에 내재되어 있는 숨겨진 요소
- NMF를 통해 원본 데이터에 포함된 잠재적인 특성을 추출할 수 있음

- 추천 시스템에서는 사용자와 상품 사이의 숨겨진 선호 관계 등을 파악하는 데 활용됨

5) **W** 행렬과 **H** 행렬의 역할

① W 행렬

- 원본 행의 데이터가 각각의 잠재 요소와 얼마나 관련되어 있는지를 나타냄
- 원본 데이터의 각 행이 잠재 요소를 얼마나 포함하는지 표현함

② H 행렬

- 잠재 요소가 원본 데이터의 특성을 어떻게 구성하는지 나타냄
- 잠재 요소와 원본 데이터의 특성 사이의 관계를 표현함

6) **NMF**의 특징

- 원본 행렬의 모든 원소가 0 이상이어야 함
- 두 개의 작은 행렬로 원본 행렬을 근사함
- 데이터에 숨겨진 잠재적인 요소를 추출할 수 있음
- 차원 축소와 데이터 압축에 활용됨
- 이미지, 텍스트, 추천 시스템 등 다양한 분야에서 사용됨

7) **NMF**의 활용 분야

① 이미지 압축 및 패턴 인식

- 이미지 데이터의 잠재적인 특징을 추출하고 압축하는 데 활용됨
- 이미지의 주요 패턴을 파악하는 데 사용됨

② 텍스트 토픽 모델링(Topic Modeling)

- 문서에 포함된 단어의 관계를 분석하여 문서의 주제를 추출함
- 문서에 숨어 있는 잠재적인 의미와 주제를 파악할 수 있음

③ 문서 유사도 및 클러스터링

- 문서의 잠재적인 특성을 추출하여 문서 사이의 유사성을 파악함
- 유사한 특징을 가진 문서들을 그룹화하는 데 활용됨

④ 추천 시스템(Recommendation System)

- 사용자와 상품 사이의 평가 데이터를 행렬 분해하여 잠재적인 요소를 추출함
- 사용자가 평가하지 않은 상품에 대한 잠재적인 선호도를 예측함
- 예측한 평가 순위를 바탕으로 사용자에게 적합한 상품을 추천함

⑤ 잠재 요인 기반 추천(Latent Factor 기반 추천)

- 사용자와 상품의 평가 데이터에 숨겨진 잠재 요소를 추출하는 방식
- 사용자가 어떤 특성을 선호하는지, 상품이 어떤 특성을 가지고 있는지 분석함

- 이를 바탕으로 사용자가 아직 평가하지 않은 상품의 평가 점수를 예측할 수 있음

8) NMF 실습에서 알아둘 내용

- 사이킷런에서는 `NMF` 클래스를 사용함
- `n_components`: 추출할 컴포넌트의 개수를 지정함
- `fit()`: 입력 데이터에 대해 학습함
- `transform()`: 학습된 행렬 분해를 바탕으로 데이터를 변환함
- 붓꽃 데이터에 NMF를 적용하면 기존 특성을 두 개의 컴포넌트로 축소하여 표현할 수 있음